Explicit Euler

First-order forward discretization

1 Numerical map

For an initial-value problem

$$\dot{z} = f(t, z), \qquad z(t_n) = z_n,$$

Taylor expansion gives

$$z(t_n + h) = z_n + hf(t_n, z_n) + \mathcal{O}(h^2).$$

Explicit Euler discards the remainder and defines

$$\Phi_{h,t_n}(z_n) = z_n + hf(t_n, z_n).$$

The method requires one vector-field evaluation, has local truncation error $\mathcal{O}(h^2)$ and, under the standard stability and smoothness assumptions, global error $\mathcal{O}(h)$.

2 Stability

For the test equation $y' = \lambda y$ the amplification factor is

$$R(\xi) = 1 + \xi, \qquad \xi = h\lambda.$$

Absolute stability requires $|1 + \xi| \leq 1$, the disk centered at -1 with radius one. The bounded stability region makes this method unsuitable for stiff problems unless the step is very small.

3 Hamiltonian interpretation

For a canonical Hamiltonian system with Jacobian $A = Df(z_n)$, the step tangent is $D\Phi = I + hA$. Even when A is Hamiltonian, $(D\Phi)^T \Omega D\Phi = \Omega + h^2 A^T \Omega A$ in general. Explicit Euler is therefore neither symmetric nor symplectic and does not preserve energy in general. It serves as a transparent first-order baseline for accuracy and geometric diagnostics.

4 Implemented configuration

One forward Euler stage. Energy tracking is optional and passive. Forward Euler evaluates the passive derivative at the start of the accepted step. The common state conventions are specified below.

5 Implementation correspondence

The method is implemented in `src/methods/classical/euler.py`. The fixed-grid execution lifecycle is documented in the sibling `simulation/` directory; shared physical dynamics are documented outside the model tree.

6 Physical accepted states and passive energy tracking

For N particles with d physical components, every accepted snapshot contains $Z \in \mathbb{R}^{dN}$, one scalar time t, and optional separate physical momentum $\kappa \in \mathbb{R}^N$. Thus GC accepted vectors have $2N$ components and FC vectors have $4N$, whether energy is tracked or not. A doubled method prepares temporary $(u, v) \in \mathbb{R}^{2dN}$ copies, allows them to separate, and projects to one physical result before acceptance. The representation stores only the physical size and never owns a clock, momentum, physical coordinate declaration, dynamics or solver.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, Z), \qquad \kappa(t_0) = 0, \qquad \Delta K(t) = H(t, Z(t)) + \kappa(t) - H(t_0, Z_0).$$

GC split-map contributions are summed in temporary work and divided by two to obtain physical momentum. Tracking leaves physical stages, nonlinear stopping scales and adaptive acceptance unchanged. An autonomous Hamiltonian supplies an explicit zero momentum derivative.

Hairer's constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA's simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired. Dynamics declare component order and position/velocity roles through immutable structural data; initial conditions and results retain those data independently. Methods own maps, projection equations and momentum updates. Generic copy operations live in `src/formulations/state.py`.

Controllers and compiled JAX loops carry physical values and momentum separately. Observers receive physical states. Energy histories in `States.diagnostics` retain their existing labels; `extended_time` has shape (N, n_{samples}) and is broadcast from scalar sample times. Fixed shadow samples leave accepted states and counters unchanged. Adaptive quadrature uses accepted dense output and physical-only error control.

7 Optional JAX execution

An optional JAX driver executes the same numerical map on one selected CPU or GPU, with particle operations batched inside a compiled sequential time loop. It reuses the canonical coefficients and prepared potential splines. Physical-state tolerance scaling, signed stage times, passive energy weights and independent shortened output maps retain their definitions above. Applicable Newton systems use independent particle blocks; an explicitly selected Broyden iteration retains its global secant matrix. Nonlinear iteration limits and failure checks remain active on the device.

Both execution routes use double precision and return the same physical solution layout. Floating-point reassociation can produce round-off differences without changing the designed order or geometric properties. First-call compilation is separate from warmed integration timings, which include transfers and output construction. The sibling runtime document and shared execution contract specify supported callbacks and exact device scope.